

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Name: Roger A Raju

Roll no: 2503a52370

Batch-13

Task 1 – Stack Implementation

1. Question

Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

2. Prompt:

Complete this Python Stack class for a beginner data structures lab. Add push(), pop(), peek(), and is_empty(), handle an empty stack, include short docstrings/comments, and add a small test section.

3. Code:

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if self.is_empty():
            return None
        return self.items.pop()

    def peek(self):
        if self.is_empty():
            return None
        return self.items[-1]

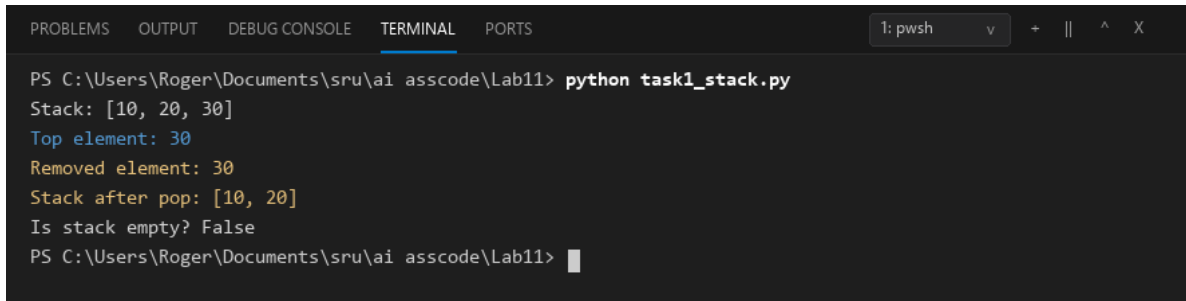
    def is_empty(self):
        return len(self.items) == 0

if __name__ == "__main__":
    stack = Stack()
    stack.push(10)
    stack.push(20)
    stack.push(30)

    print("Stack:", stack.items)
    print("Top element:", stack.peek())
    print("Removed element:", stack.pop())
```

```
print("Stack after pop:", stack.items)
print("Is stack empty?", stack.is_empty())
```

4. Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh v + || ^ X
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task1_stack.py
Stack: [10, 20, 30]
Top element: 30
Removed element: 30
Stack after pop: [10, 20]
Is stack empty? False
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> █
```

5. Final Justification

The Stack was implemented successfully with the required methods. It follows the LIFO principle and was tested with sample operations.

Task 2 – Queue Implementation

1. Question

Use AI to implement a Queue using Python lists.

2. Prompt:

Complete this Python Queue class using a list. Add enqueue(), dequeue(), peek(), and size(), follow FIFO, handle an empty queue, and add short comments plus a small test section.

3. Code:

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.size() == 0:
            return None
        return self.items.pop(0)

    def peek(self):
        if self.size() == 0:
            return None
        return self.items[0]

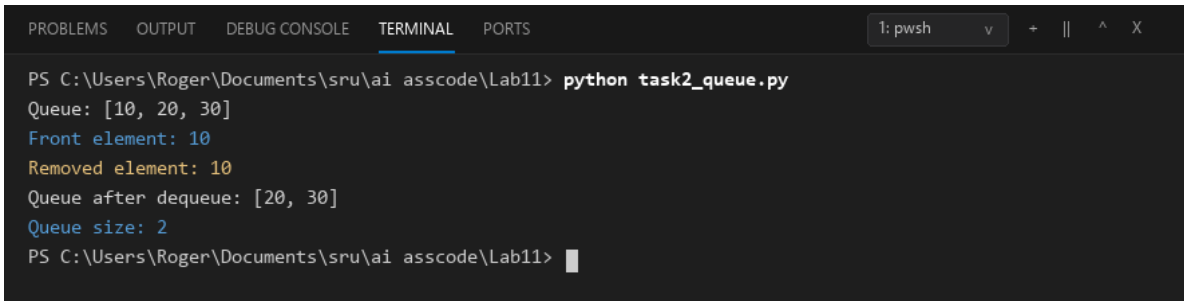
    def size(self):
        return len(self.items)

if __name__ == "__main__":
```

```
queue = Queue()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)

print("Queue:", queue.items)
print("Front element:", queue.peek())
print("Removed element:", queue.dequeue())
print("Queue after dequeue:", queue.items)
print("Queue size:", queue.size())
```

4. Output :



```
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task2_queue.py
Queue: [10, 20, 30]
Front element: 10
Removed element: 10
Queue after dequeue: [20, 30]
Queue size: 2
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11>
```

5. Final Justification

The Queue was implemented using a Python list and tested with sample operations. It follows the FIFO principle as required.

Task 3 – Linked List

1. Question

Use AI to generate a Singly Linked List with insert and display methods.

2. Prompt:

Create a beginner-friendly singly linked list in Python with Node and LinkedList classes. Add insert() and display(), with short comments/docstrings and a small test section.

3. Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
```

```

        return
    current = self.head
    while current.next is not None:
        current = current.next
    current.next = new_node

def display(self):
    current = self.head
    while current is not None:
        print(current.data, end=" -> ")
        current = current.next
    print("None")

if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)

    print("Linked List:")
    linked_list.display()

```

4. Output :

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task3_linked_list.py
Linked List:
10 -> 20 -> 30 -> None
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11>

```

5. Final Justification

The singly linked list was implemented with nodes connected through next references. Insert and display operations were tested successfully.

Task 4 – Binary Search Tree (BST)

1. Question

Use AI to create a BST with insert and in-order traversal methods.

2. Prompt:

Create a beginner-friendly Python BST with recursive insert() and in-order traversal. Add short comments/docstrings and test it with sample values.

3. Code:

```

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None

```

```

        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, data):
        self.root = self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        if node is None:
            return Node(data)
        if data < node.data:
            node.left = self._insert_recursive(node.left, data)
        elif data > node.data:
            node.right = self._insert_recursive(node.right, data)
        return node

    def in_order(self):
        self._in_order_recursive(self.root)
        print()

    def _in_order_recursive(self, node):
        if node is not None:
            self._in_order_recursive(node.left)
            print(node.data, end=" ")
            self._in_order_recursive(node.right)

if __name__ == "__main__":
    tree = BST()
    values = [50, 30, 70, 20, 40, 60, 80]
    for value in values:
        tree.insert(value)

    print("In-order traversal:")
    tree.in_order()

```

4. Output :

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task4_bst.py
In-order traversal:
20 30 40 50 60 70 80
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11>

```

5. Final Justification

The BST was implemented using recursive insertion and in-order traversal. The traversal produced the values in sorted order.

Task 5 – Priority Queue

1. Question

Use AI to implement a priority queue using Python's heapq module.

2. Prompt:

Create a simple Python PriorityQueue using heapq. Implement enqueue(priority), dequeue(), and display(), handle an empty queue, and add short comments plus tests.

3. Code:

```
import heapq

class PriorityQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item, priority):
        heapq.heappush(self.queue, (-priority, item))

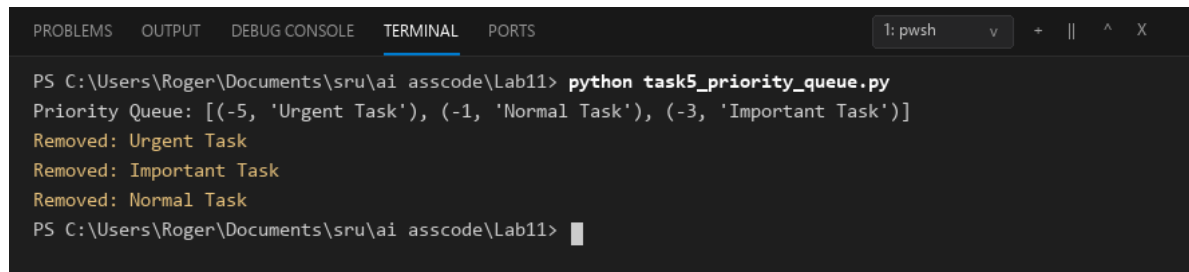
    def dequeue(self):
        if len(self.queue) == 0:
            return None
        priority, item = heapq.heappop(self.queue)
        return item

    def display(self):
        print("Priority Queue:", self.queue)

if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Normal Task", 1)
    pq.enqueue("Important Task", 3)
    pq.enqueue("Urgent Task", 5)

    pq.display()
    print("Removed:", pq.dequeue())
    print("Removed:", pq.dequeue())
    print("Removed:", pq.dequeue())
```

4. Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh v + || ^ X
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task5_priority_queue.py
Priority Queue: [(-5, 'Urgent Task'), (-1, 'Normal Task'), (-3, 'Important Task')]
Removed: Urgent Task
Removed: Important Task
Removed: Normal Task
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11>
```

5. Final Justification

The Priority Queue was implemented using Python's `heapq` module. Items were inserted and removed according to their assigned priority.

Task 6 – Deque

1. Question

Use AI to implement a double-ended queue using `collections.deque`.

2. Prompt:

Create a beginner-friendly DequeDS class using `collections.deque`. Implement insertion and removal from both ends, `display()`, empty handling, and short comments/docstrings.

3. Code:

```
from collections import deque

class DequeDS:
    def __init__(self):
        self.items = deque()

    def insert_front(self, item):
        self.items.appendleft(item)

    def insert_rear(self, item):
        self.items.append(item)

    def remove_front(self):
        if len(self.items) == 0:
            return None
        return self.items.popleft()

    def remove_rear(self):
        if len(self.items) == 0:
            return None
        return self.items.pop()

    def display(self):
        print("Deque:", list(self.items))

if __name__ == "__main__":
    d = DequeDS()
    d.insert_front(10)
    d.insert_rear(20)
    d.insert_front(5)

    d.display()
    print("Removed from front:", d.remove_front())
    print("Removed from rear:", d.remove_rear())
    d.display()
```

4. Output :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh v + || ^ X
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task6_deque.py
Deque: [5, 10, 20]
Removed from front: 5
Removed from rear: 20
Deque: [10]
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> █
```

5. Final Justification

The Deque was implemented using `collections.deque` and tested from both ends. It supports insertion and removal at the front and rear.

Task 7 – Real-Time Application Challenge – Choose the Right Data Structure

1. Question

For the Campus Resource Management System, select a suitable data structure for each feature, justify each choice briefly, and implement one feature.

2. Prompt:

For the five Campus Resource Management features, choose a suitable data structure and give a short justification for each. Then implement one selected feature as a simple Python program with comments.

3. Code:

```
class CafeteriaQueue:
    def __init__(self):
        self.orders = []

    def add_order(self, student_name, food):
        order = f"{student_name} - {food}"
        self.orders.append(order)
        print("Order added:", order)

    def serve_order(self):
        if len(self.orders) == 0:
            print("No orders to serve.")
            return
        order = self.orders.pop(0)
        print("Order served:", order)

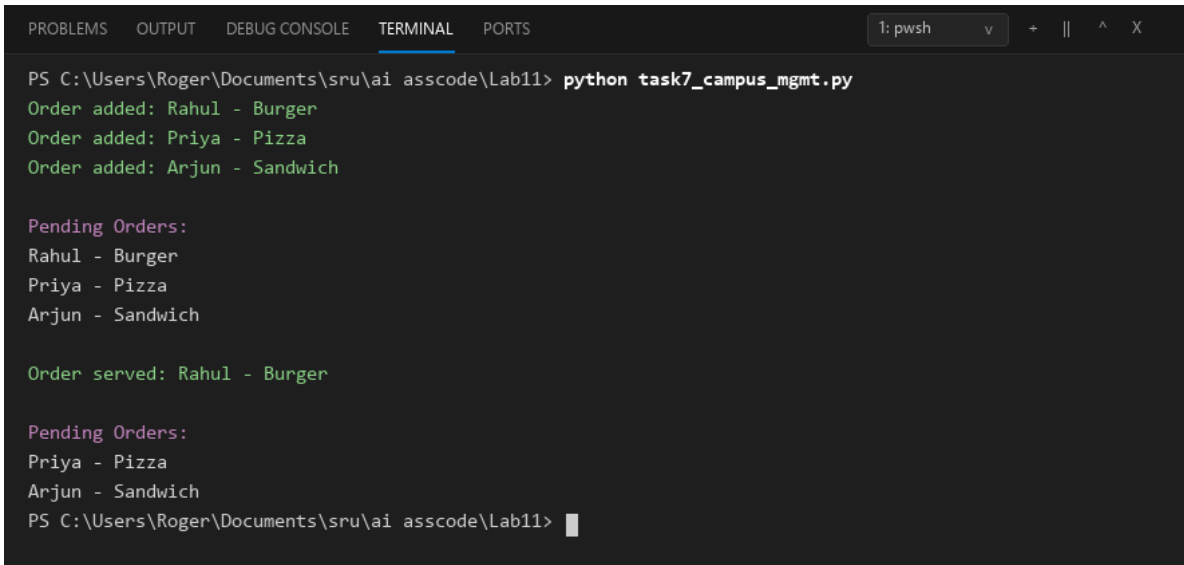
    def display_orders(self):
        if len(self.orders) == 0:
            print("No pending orders.")
        else:
            print("Pending Orders:")
            for order in self.orders:
                print(order)

if __name__ == "__main__":
    cafeteria = CafeteriaQueue()
```

```
cafeteria.add_order("Rahul", "Burger")
cafeteria.add_order("Priya", "Pizza")
cafeteria.add_order("Arjun", "Sandwich")

print()
cafeteria.display_orders()
print()
cafeteria.serve_order()
print()
cafeteria.display_orders()
```

4. Output :



```
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task7_campus_mgmt.py
Order added: Rahul - Burger
Order added: Priya - Pizza
Order added: Arjun - Sandwich

Pending Orders:
Rahul - Burger
Priya - Pizza
Arjun - Sandwich

Order served: Rahul - Burger

Pending Orders:
Priya - Pizza
Arjun - Sandwich
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11>
```

5. Final Justification

Suitable data structures were selected for the campus features based on their required operations. One feature was implemented as a working Python program with AI assistance.

Task 8 – Smart E-Commerce Platform – Data Structure Challenge

1. Question

For the Smart Online Shopping System, select a suitable data structure for each feature, justify each choice briefly, and implement one feature.

2. Prompt:

For the five e-commerce features, choose a suitable data structure and give a short justification for each. Then implement one selected feature as a simple Python program with comments.

3. Code:

```
class OrderQueue:
    def __init__(self):
        self.orders = []

    def add_order(self, order_id, customer):
```

```

        order = f"{order_id} - {customer}"
        self.orders.append(order)
        print("Order added:", order)

    def process_order(self):
        if len(self.orders) == 0:
            print("No orders to process.")
            return
        order = self.orders.pop(0)
        print("Order processed:", order)

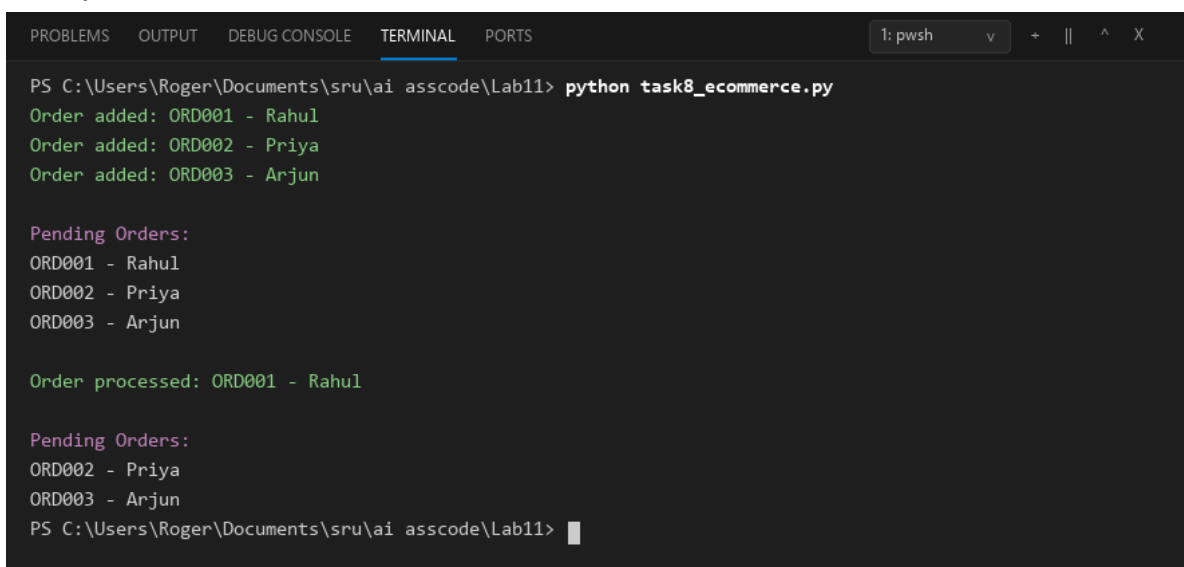
    def display_orders(self):
        if len(self.orders) == 0:
            print("No pending orders.")
        else:
            print("Pending Orders:")
            for order in self.orders:
                print(order)

if __name__ == "__main__":
    order_queue = OrderQueue()
    order_queue.add_order("ORD001", "Rahul")
    order_queue.add_order("ORD002", "Priya")
    order_queue.add_order("ORD003", "Arjun")

    print()
    order_queue.display_orders()
    print()
    order_queue.process_order()
    print()
    order_queue.display_orders()

```

4. Output :



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai asscode\Lab11> python task8_ecommerce.py
Order added: ORD001 - Rahul
Order added: ORD002 - Priya
Order added: ORD003 - Arjun

Pending Orders:
ORD001 - Rahul
ORD002 - Priya
ORD003 - Arjun

Order processed: ORD001 - Rahul

Pending Orders:
ORD002 - Priya
ORD003 - Arjun
PS C:\Users\Roger\Documents\sru\ai asscode\Lab11>

```

5. Final Justification

Suitable data structures were selected for the e-commerce features according to their operations. One feature was implemented as a working Python program with AI assistance.

Task 7 – Campus Resource Management: Data Structure Selection

Campus Feature	Data Structure	Reason
Student Attendance Tracking	Queue	Students can be processed in the order they enter the campus.
Event Registration	Hash Table	It provides fast search using a student or participant ID.
Library Book Borrowing	Priority Queue	Borrowing requests can be handled according to due dates or priority.
Bus Scheduling	Graph	Bus stops and routes can be represented as connected nodes and edges.
Cafeteria Order Queue	Queue	Orders are served in the same order in which students place them.

Task 8 – Smart E-Commerce Platform: Data Structure Challenge

E-Commerce Feature	Data Structure	Reason
Shopping Cart Management	Deque	Products can be added or removed efficiently from either end.
Order Processing System	Queue	Orders are processed in the same order in which they are placed.
Top-Selling Products Tracker	Priority Queue	Products can be handled according to their sales priority.
Product Search Engine	Hash Table	Product IDs can be used for fast lookup.
Delivery Route Planning	Graph	Warehouses and delivery locations can be represented as connected nodes.